

aflock: Cryptographically Signed Policies for Constrained AI Agent Execution

Cole Kennedy*
TestifySec, Inc.

January 2026

Abstract

As AI agents become increasingly autonomous—executing commands, accessing APIs, spawning sub-agents, and managing sensitive data—ensuring they operate within authorized bounds becomes critical. We present **aflock**, a policy framework that cryptographically constrains AI agent behavior through signed policy files, verifiable attestations, and cross-step constraint verification. Unlike existing approaches that rely on runtime guardrails or trusted execution environments, aflock provides a software-only solution inspired by workload identity systems (SPIFFE/SPIRE) and software supply chain security (in-toto). Key innovations include: (1) agent identity derivation from introspectable properties (model, environment, tools) via MCP socket inspection, (2) separation of JWT-based authorization from server-held signing keys, (3) session merkle trees for ordering and completeness proofs, (4) sublayouts for hierarchical sub-agent delegation with attenuated constraints, and (5) cross-step Rego evaluation for cumulative limit enforcement. We position aflock within the emerging landscape of agent security frameworks and outline a verification algorithm that enables post-hoc compliance auditing with cryptographic guarantees.

1 Introduction

The proliferation of autonomous AI agents presents a fundamental security challenge: how do principals (humans, organizations, or parent agents) constrain what an agent can do, prove it operated within bounds, and verify compliance after the fact?

Consider a software development agent tasked with implementing a feature. It may read files, execute shell commands, make API calls, and even spawn sub-agents for research tasks. Without constraints, such an agent could:

- Exceed cost budgets through unbounded API usage
- Access sensitive files (credentials, environment variables)
- Execute dangerous commands (force pushes, destructive operations)
- Exfiltrate data through unrestricted network access
- Delegate unconstrained authority to sub-agents

Existing solutions address fragments of this problem. Runtime guardrails [10] filter inputs and outputs but provide no cryptographic proof of compliance. Trusted execution environments [1] offer strong isolation but require specialized hardware. Identity frameworks [5] authenticate workloads but weren’t designed for non-deterministic AI agents.

We present **aflock**, a policy framework that treats agent constraints like dependency locks treat package versions—as signed, immutable specifications that the agent cannot modify. The name derives from “agent flow lock,” analogous to how `package-lock.json` locks dependencies in software projects.

*Corresponding author. Email: cole@testifysec.com

1.1 Contributions

This paper makes the following contributions:

1. **Policy Schema:** A comprehensive JSON schema for expressing agent constraints including resource limits, tool allowlists, file access patterns, and evaluator specifications.
2. **Identity Derivation:** A mechanism for deriving agent identity from introspectable properties (model, environment, tools, policy digest) via MCP socket inspection, inspired by SPIRE’s workload attestation.
3. **Attestation Model:** An in-toto-compatible attestation format for agent actions that enables cryptographic verification of constraint compliance.
4. **Cross-Step Verification:** A Rego-based evaluation framework that accesses attestations across all execution steps for cumulative constraint checking.
5. **Sublayout Delegation:** A hierarchical model for sub-agent policy delegation with mandatory constraint attenuation, inspired by in-toto sublayouts.

2 Threat Model and Assumptions

2.1 Threat Model

We consider an adversarial agent that attempts to:

- **Exceed resource limits:** Consume more tokens, API calls, or cost than authorized
- **Access unauthorized resources:** Read/write files, call APIs, or access secrets outside the allowlist
- **Forge attestations:** Create false records of compliant behavior
- **Escalate privileges:** Spawn sub-agents with greater authority than the parent
- **Tamper with ordering:** Reorder, omit, or inject execution steps

2.2 Trust Assumptions

1. **aflock server integrity:** The aflock server (analogous to a SPIRE server) is trusted and controls signing keys
2. **Policy signer authenticity:** Policies are signed by authorized principals (humans or trusted systems)
3. **Transport security:** MCP communication occurs over authenticated channels
4. **Process isolation:** The OS enforces process boundaries (agents cannot inspect other processes’ memory)

We explicitly **do not** assume:

- Trusted execution environments (TEE)
- Hardware security modules (HSM)
- Deterministic agent behavior
- Agent honesty or cooperation

3 System Architecture

Figure 1 illustrates the aflock architecture. The system consists of three primary components: the AI agent, the aflock server, and the verification layer.

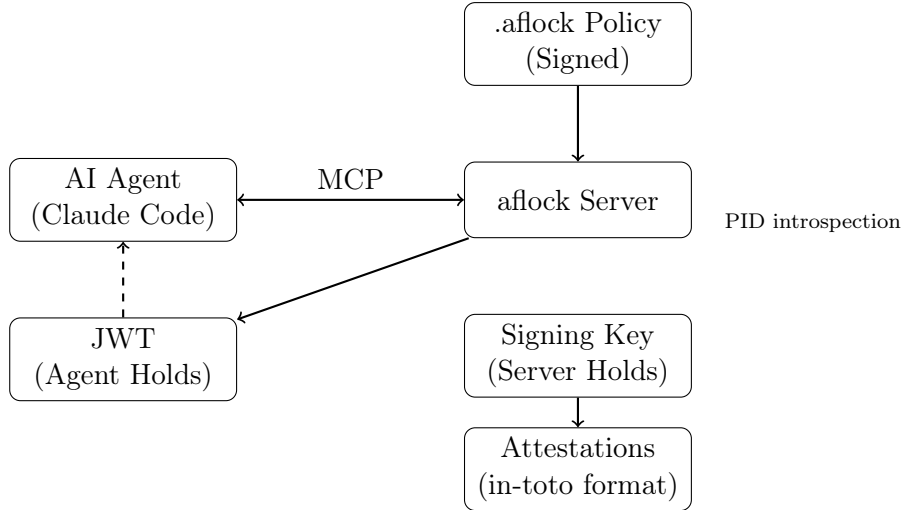


Figure 1: aflock architecture showing separation between agent-held JWT and server-held signing key. The agent communicates via MCP; the server derives identity through PID introspection.

3.1 Agent Identity Derivation

A core insight of aflock is that agent identity should be *derived* from introspectable properties rather than *assigned*. When an agent connects to the aflock server via MCP over a Unix domain socket, the server can obtain the connecting process’s PID through `SO_PEERCREC` (Linux) or `LOCAL_PEERCREC` (macOS).

From the PID, the server introspects:

- **Binary path:** The executable running the agent
- **User/Group ID:** Process ownership
- **Container ID:** If running in a container (via `/proc/PID/cgroup`)
- **Environment variables:** Model configuration, session identifiers

Combined with information from the MCP handshake (model name, available tools), the server computes:

$$\text{AgentIdentity} = \text{SHA256}(\text{model} \parallel \text{env} \parallel \text{tools} \parallel \text{policyDigest} \parallel \text{parent}) \quad (1)$$

This approach mirrors SPIRE’s workload attestation [5], where identity emerges from verifiable properties rather than self-declaration. The agent *cannot lie about its identity* because the server independently verifies all components.

3.2 JWT and Signing Key Separation

A critical security property is the separation between authorization (what the agent is allowed to do) and attestation (proof of what the agent did). The agent receives a short-lived JWT containing:

```

{
  "sub": "sha256:a1b2c3...",
  "iat": 1737200000,
  "exp": 1737203600,
  "grants": {
    "tools": ["Read", "Edit", "Bash"],
    "files": {"allow": ["src/**"], "deny": ["**/.env"]},
    "limits": {"maxSpendUSD": 10.00}
  }
}

```

```
}  
}
```

The agent presents this JWT when requesting actions, but *all attestations are signed by a key the agent never sees*. This ensures that even a compromised agent cannot forge attestations claiming compliant behavior.

3.3 MCP Integration

The Model Context Protocol (MCP) provides the communication channel between agent and server. The aflock server exposes MCP tools for:

- **aflock_authorize**: Request authorization for an action
- **aflock_attest**: Record an action (server signs attestation)
- **aflock_check_limits**: Query remaining budget
- **aflock_delegate**: Create sublayout for sub-agent

This design allows aflock to integrate with any MCP-compatible agent (Claude Code, custom agents) without modification to the agent's core logic.

4 Policy Schema

The `.aflock` policy file is a signed JSON document specifying all constraints on agent behavior. We describe the key sections.

4.1 Identity Constraints

```
{  
  "identity": {  
    "allowedModels": ["claude-opus-4-5-20251101", "claude-sonnet-4-*"],  
    "allowedEnvironments": ["container:ghcr.io/org/*", "user:deploy-*"],  
    "requiredTools": ["Read", "Edit"]  
  }  
}
```

Identity constraints specify which agent configurations are authorized to operate under this policy. Glob patterns enable flexible matching while maintaining security.

4.2 Resource Limits

```
{  
  "limits": {  
    "maxSpendUSD": {"value": 10.00, "enforcement": "fail-fast"},  
    "maxTokensIn": {"value": 500000, "enforcement": "fail-fast"},  
    "maxTurns": {"value": 50, "enforcement": "post-hoc"},  
    "maxWallTimeSeconds": {"value": 3600, "enforcement": "fail-fast"}  
  }  
}
```

Each limit specifies an enforcement mode:

- **fail-fast**: Abort immediately when breached (for cost, security)
- **post-hoc**: Verify at completion (for quality, turn count)

4.3 Grants and Allowlists

```
{
  "tools": {
    "allow": ["Read", "Edit", "Bash", "Glob", "Grep"],
    "deny": ["Task"],
    "requireApproval": ["Bash:rm -rf *", "Bash:git push --force"]
  },
  "files": {
    "allow": ["src/**", "tests/**"],
    "deny": ["**/.env", "**/secrets/**"],
    "readOnly": ["package.json", "go.mod"]
  },
  "grants": {
    "secrets": {"allow": ["vault:secret/data/readonly/*"]},
    "apis": {"allow": ["https://api.anthropic.com/*"]}
  }
}
```

The allowlist model follows the principle of least privilege: all access is denied unless explicitly granted.

4.4 Materials Binding

```
{
  "materialsFrom": {
    "session": {
      "path": "${CLAUDE_SESSION_PATH}",
      "algorithm": "sha256",
      "merkleTree": true
    },
    "git": {
      "treeHash": "${GIT_TREE_HASH}"
    }
  }
}
```

The `materialsFrom` directive binds the policy to external artifacts. The session merkle tree is particularly important: by constructing a merkle tree over the session JSONL, we can prove:

- **Order:** Turn n came after turns 0 through $n - 1$
- **Completeness:** No turns were omitted
- **Distance:** Turns are contiguous (no gaps)

4.5 Evaluators

aflock supports two types of evaluators for constraint verification:

4.5.1 Rego Evaluators

```
package aflock
import rego.v1

turns := [t | some t in input.attestations]
sum_spend := sum([t.predicate.metrics.costUSD | some t in turns])

deny contains msg if {
```

```

sum_spend > input.policy.limits.maxSpendUSD.value
msg := sprintf("Spend $%.2f exceeds $%.2f",
               [sum_spend, input.policy.limits.maxSpendUSD.value])
}

```

Rego policies enable cross-step verification: the evaluator receives all attestations and can compute cumulative metrics.

4.5.2 AI Evaluators

```

{
  "evaluators": {
    "ai": [{
      "name": "quality-check",
      "prompt": "PASS if code is production-ready. FAIL otherwise.",
      "model": "claude-opus-4-5-20251101"
    }]
  }
}

```

AI evaluators assess qualitative properties that are difficult to express in formal logic, such as code quality or task completion.

5 Sublayouts: Hierarchical Delegation

Inspired by in-toto sublayouts [6], aflock supports hierarchical sub-agent delegation with mandatory constraint attenuation.

```

{
  "sublayouts": [{
    "name": "research-agent",
    "policy": "./policies/research.aflock",
    "limits": {"maxSpendUSD": {"value": 2.00}},
    "inherit": ["domains", "functionaries"]
  }]
}

```

The following invariants are enforced:

1. **Attenuation:** Sub-agent limits must be $\leq$ parent limits
2. **Accumulation:** Sub-agent spend counts toward parent totals
3. **Namespacing:** Sub-agent attestations are prefixed with sublayout name
4. **Recursion:** Verification recurses into sublayouts

6 Verification Algorithm

The verification procedure for an aflock-constrained execution proceeds in six phases:

7 Related Work

Table 1 compares aflock to related systems across key capabilities.

Table 1: Comparison of AI agent constraint systems

Capability	Omega	VeriGuard	ARIA	ShieldAgent	aflock
Signed policy file	?	×	×	×	✓
Identity derivation	TEE	×	DID	×	PID+MCP
Cryptographic attestations	✓	×	VC	×	in-toto
Cross-step verification	?	×	×	×	Rego
Spend/token limits	×	×	×	×	✓
Sub-agent delegation	?	×	✓	×	sublayouts
Session ordering proofs	×	×	×	×	Merkle
Software-only (no TEE)	×	✓	✓	✓	✓

7.1 Trusted Execution Approaches

Omega [1] provides strong isolation guarantees through confidential VMs and GPUs (AMD SEV-SNP, NVIDIA H100). While offering hardware-backed security, this approach requires specialized infrastructure unavailable in many deployment contexts. aflock provides a software-only alternative with cryptographic (rather than hardware) guarantees.

7.2 Formal Verification

VeriGuard [2] applies formal verification to agent policies using Hoare triples, proving safety properties before execution. This approach complements aflock: VeriGuard could verify policy correctness while aflock provides runtime attestation that the policy was followed.

7.3 Identity Frameworks

SPIFFE/SPIRE [5] provides the architectural inspiration for aflock’s identity derivation. However, SPIFFE treats workload replicas as identical—appropriate for deterministic microservices but problematic for non-deterministic AI agents. aflock extends this model with policy-aware identity binding.

ARIA [9] proposes a graph-native identity model with DIDs and verifiable credentials. While powerful for complex delegation scenarios, ARIA’s graph-based approach adds complexity that may be unnecessary for simpler agent deployments.

7.4 Runtime Guardrails

ShieldAgent [3] and AgentGuardian [4] provide runtime policy enforcement but lack cryptographic attestation. These systems cannot prove compliance after the fact—a critical requirement for regulated industries.

7.5 Token-Based Delegation

Google’s Macaroons [7] and Biscuit tokens [8] provide elegant primitives for offline attenuation and delegation. aflock could adopt these as transport mechanisms while providing the higher-level policy semantics and attestation framework.

7.6 Cost Management

Google’s Budget Tracker [11] injects budget awareness into agent reasoning loops. aflock’s limit enforcement provides similar functionality with the addition of cryptographic proof that limits were respected.

8 Discussion

8.1 Limitations

Trust in aflock server: The aflock server holds signing keys and must be trusted. A compromised server could forge attestations. Mitigations include HSM-backed keys and multi-party signing.

MCP dependency: aflock requires MCP-compatible agents. Agents using other protocols require adapters.

AI evaluator reliability: AI-based quality checks are probabilistic. Critical constraints should use Rego policies for deterministic verification.

Performance overhead: Attestation signing adds latency to each action. Batching strategies can mitigate this for high-frequency operations.

8.2 Future Work

Distributed verification: Enabling third parties to verify attestations without accessing the full execution trace.

Policy inheritance: Allowing policies to extend base templates for organizational standardization.

Real-time monitoring: Streaming attestations to monitoring systems for live compliance dashboards.

Formal semantics: Developing a formal semantics for the policy language to enable policy analysis and composition verification.

9 Conclusion

aflock provides a comprehensive framework for constraining AI agent behavior through cryptographically signed policies and verifiable attestations. By combining insights from workload identity (SPIFFE/SPIRE), software supply chain security (in-toto), and capability-based authorization (Macaroons), aflock enables principals to define, enforce, and verify agent constraints without requiring trusted hardware.

As AI agents become more autonomous and widely deployed, frameworks like aflock will be essential for maintaining human oversight and regulatory compliance. The separation of authorization (JWT) from attestation (server-signed) ensures that even compromised agents cannot forge evidence of compliant behavior.

We release the aflock specification and examples at <https://github.com/aflock-ai/aflock> to enable community feedback and adoption.

References

- [1] Author et al. Omega: Trusted AI Agents in the Cloud. *arXiv preprint arXiv:2512.05951*, 2025.
- [2] Lesly Miculicich et al. VeriGuard: Enhancing LLM Agent Safety via Verified Code Generation. *arXiv preprint arXiv:2510.05156*, 2025.
- [3] Zhaorun Chen, Mintong Kang, Bo Li et al. ShieldAgent: Shielding Agents via Verifiable Safety Policy Reasoning. *arXiv preprint arXiv:2503.22738*, 2025.
- [4] Nadya Abaev, Denis Klimov et al. AgentGuardian: Learning Access Control Policies to Govern AI Agent Behavior. *arXiv preprint arXiv:2601.10440*, 2026.

- [5] CNCF. SPIFFE: Secure Production Identity Framework For Everyone. <https://spiffe.io/>, 2024.
- [6] Santiago Torres-Arias et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.
- [7] Arnar Birgisson et al. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [8] Geoffroy Couprie. Biscuit: Authorization tokens with decentralized verification. <https://www.biscuitsec.org/>, 2024.
- [9] ISACA. The Looming Authorization Crisis: Why Traditional IAM Fails Agentic AI. *ISACA Industry News*, 2025.
- [10] Guardrails AI. Guardrails: Adding guardrails to large language models. <https://www.guardrailsai.com/>, 2024.
- [11] Google Research. Budget Tracker and BATS Framework for AI Agent Cost Control. *Google AI Blog*, 2025.

A Full Policy Schema

The complete JSON Schema for .aflock files is available at: <https://github.com/aflock-ai/aflock/blob/main/docs/spec.md>

B Example: Compliance Control Evaluation

Listing 2: OSCAL compliance evaluation policy

```
{
  "version": "1.0",
  "name": "oscal-control-evaluation",
  "expires": "2026-06-01T00:00:00Z",

  "identity": {
    "allowedModels": ["claude-opus-4-5-*"],
    "allowedEnvironments": ["container:ghcr.io/testifysec/*"]
  },

  "limits": {
    "maxSpendUSD": {"value": 25.00, "enforcement": "fail-fast"},
    "maxTokensIn": {"value": 1000000, "enforcement": "post-hoc"}
  },

  "tools": {
    "allow": ["Read", "Write", "Glob", "Grep", "Bash", "WebFetch"],
    "deny": ["Task"]
  },

  "evaluators": {
    "rego": [{
      "name": "all-controls-assessed",
      "policy": "deny contains msg if { count(missing_controls) > 0 }"
    }],
  },
}
```

```
    "ai": [{
      "name": "assessment-quality",
      "prompt": "PASS if each control assessment includes evidence
        references and remediation guidance. FAIL otherwise.",
      "model": "claude-opus-4-5-20251101"
    }]
  },
  "sublayouts": [{
    "name": "evidence-collector",
    "policy": "./policies/evidence.aflock",
    "limits": {"maxSpendUSD": {"value": 5.00}}
  }]
}
```

Listing 1: aflock Verification Algorithm

```
VERIFY(Policy P, Attestations A, Materials M):

  Phase 1: Signature Verification
  for each attestation a_i in A:
    if not VerifySignature(a_i, P.functionaries):
      return FAIL: "Invalid signature on attestation i"

  Phase 2: Identity Verification
  for each attestation a_i in A:
    if not MatchesIdentity(a_i.agent, P.identity):
      return FAIL: "Identity mismatch on attestation i"

  Phase 3: Materials Binding
  if P.materialsFrom.session.merkleTree:
    root = ComputeMerkleRoot(M.session)
    if root != A[n].predicate.sessionRoot:
      return FAIL: "Session merkle root mismatch"

  Phase 4: Constraint Evaluation
  for each Rego evaluator e in P.evaluators.rego:
    result = EvaluateRego(e.policy, {P, A, M})
    if result.deny is not empty:
      return FAIL: result.deny

  Phase 5: AI Evaluation
  for each AI evaluator e in P.evaluators.ai:
    result = EvaluateAI(e.prompt, e.model, M)
    if result == FAIL:
      return FAIL: "AI evaluator 'e.name' failed"

  Phase 6: Sublayout Recursion
  for each sublayout s in P.sublayouts:
    A_s = {a in A : a.prefix == s.name}
    result = VERIFY(s.policy, A_s, M)
    if result == FAIL:
      return FAIL: "Sublayout 's.name' failed"

  return PASS
```